

Crewly AI Implementation Playbook

Table of Contents

1. Prompt Generation Philosophy
2. Complete Prompt Roadmap
3. Prompt Structure Standard
4. Repository Workflow
5. Phase Workflow
6. Module Workflow
7. Quality Gates
8. Human Review Guide
9. Antigravity Interaction Rules
10. Risk Management
11. Final Prompt Roadmap
12. Project Completion Workflow

1. Prompt Generation Philosophy

The methodology governing the interaction with the Google Antigravity AI coding agent requires absolute adherence to the concept of a permanently frozen architectural state. The prompt generation philosophy dictates that the AI must be treated exclusively as a high-velocity execution engine, never as a system architect. Every implementation prompt generated from this playbook must explicitly bound the AI within the uncompromising constraints of the Enterprise SaaS Blueprint v3 and the Master Project Prompt (Prompt 00).¹

The core philosophy demands zero-hallucination execution, meaning the AI is strictly forbidden from inventing business rules, assuming missing external dependencies, proposing alternative software frameworks, or optimizing the pre-approved architecture.¹

To achieve deterministic output expectations at an enterprise scale, the prompt generation process enforces a strict separation between the permanent project context and the module-specific implementation instructions. Prompt 00 serves as the perpetual context window anchor. It ensures the AI consistently applies global architectural principles, primarily the "build like a suite, launch like a wedge" product strategy, the free-tier first design imperative, and the absolute requirement for vendor independence.¹ Subsequent module prompts will only supply the specific eighteen-point design for the module currently under construction.

Furthermore, the philosophy enforces a strict referencing model across the entire platform. The AI must be repeatedly instructed that shared entities within the Core Directory—such as the Employee, the Structure Node, or the CRM Contact—are accessed solely via immutable internal identifiers and must never be duplicated across module boundaries.¹ All prompts must instruct the AI to treat the data architecture as non-negotiable. This is particularly vital regarding the centralized implementation of PostgreSQL Row-Level Security (RLS) as the

primary multi-tenancy isolation boundary, the secondary application of Role-Based Access Control (RBAC), and the absolute, system-wide prohibition of hard deletes in favor of status-based lifecycle transitions.¹ If any instruction within a module's eighteen-point design appears ambiguous to the AI, the prompt philosophy dictates a "fail closed" response; the AI must halt generation and flag the ambiguity rather than improvising a solution.¹

2. Complete Prompt Roadmap

The development sequence for the Crewly platform is entirely linear, heavily interdependent, and strictly unalterable. The roadmap dictates that no module may be constructed in parallel with another, and no subsequent prompt may be issued until the prior module has passed all quality gates, automated testing, and human reviews. The complete execution order ensures the Platform Foundation is fully established before the MVP Business Modules introduce downstream dependencies, which in turn must be stable before the Roadmap Business Modules and Shared Platform Capabilities are generated.¹

Sequence	Prompt Designation	Phase Classification	Module / Operational Scope
1	Prompt 00	Permanent Context	Master Project Prompt (Loaded continuously for all future states)
2	Prompt 00A	Repository Initialization	Development Foundation, CI/CD, Environments, Tooling
3	Prompt 01a	Platform Foundation	Authentication (Module 1a)
4	Prompt 01b	Platform Foundation	Organization Management (Module 1b)
5	Prompt 01c	Platform Foundation	Role-Based Access Control (Module 1c)
6	Prompt 02	Platform Foundation	Subscription & Billing Engine

			(Module 2)
7	Prompt 03	Platform Foundation	Notification Framework (Module 3)
8	Prompt 04	Platform Foundation	Audit Logging (Module 4)
9	Prompt 05	Platform Foundation	File Storage (Module 5)
10	Prompt 06	Business Module (MVP)	Employee Management (Module 6)
11	Prompt 07	Business Module (MVP)	Attendance (Module 7)
12	Prompt 08	Business Module (MVP)	Leave Management (Module 8)
13	Prompt 09	Business Module (MVP)	Payroll (Module 9 - MVP Cutoff / Launchable Suite)
14	Prompt 10	Business Module (Roadmap)	Customer Relationship Management (Module 10)
15	Prompt 11	Business Module (Roadmap)	Projects (Module 11)
16	Prompt 12	Business Module (Roadmap)	Help Desk (Module 12)
17	Prompt 13	Business Module (Roadmap)	Inventory (Module 13)

18	Prompt 14	Business Module (Roadmap)	Finance (Module 14)
19	Prompt 15	Business Module (Roadmap)	Analytics & Reports (Module 15)
20	Prompt 16	Shared Capability	Platform AI Layer (Module 16)
21	Prompt 17	Shared Capability	Marketplace (Module 17)
22	Prompt 18	Platform Finalization	Final Integration & Cross-Module Contract Testing
23	Prompt 19	Platform Finalization	Penetration Testing & RLS/Isolation Validation
24	Prompt 20	Platform Finalization	Deployment Staging & Production Launch

3. Prompt Structure Standard

Every prompt delivered to the Google Antigravity agent must adhere to a rigid, standardized structure to prevent ambiguity, manage token context windows efficiently, and ensure consistent code generation patterns across the multi-month development lifecycle. The anatomy of a perfect implementation prompt begins with the system context preamble. This preamble invokes Prompt 00 to remind the AI of its role, the frozen technology stack (React.js, Vite, Tailwind CSS, Node.js, Supabase), and the inviolable architectural constraints.¹

Following the system context, the prompt must supply a highly specific task definition that translates the eighteen-point Blueprint design into actionable engineering directives.¹ This section maps out the module's exact purpose, the business problems it solves, the specific user roles involved, and the exhaustive list of business and validation rules. The prompt structure must explicitly delineate architecture boundaries by listing which existing modules the current task will consume and which future modules will eventually consume the current task's outputs. This prevents the AI from generating parallel logic for shared concerns, such as

bypassing the central notification dispatcher or creating unauthorized secondary tracking mechanisms.¹

An input schema section must follow, detailing the exact data payloads, application programming interfaces, and Supabase PostgreSQL database migrations required for the module. Finally, the expected output format must dictate the precise folder structures, file naming conventions, frontend component hierarchies, and testing coverage requirements. This section must be paired with absolute rejection criteria that define the exact conditions—such as a missing `organization_id` column or a direct call to a third-party SDK—under which the human reviewer will immediately discard the generated codebase.¹

4. Repository Workflow

The code repository workflow is engineered to maintain pristine isolation between the AI's generated output and the verified main branch, utilizing GitHub as the continuous integration and continuous deployment (CI/CD) substrate.¹ The architecture dictates a single monolithic repository housing both the React.js frontend and the Node.js/Supabase backend to minimize infrastructure complexity and adhere to the free-tier first philosophy; a microservices split is explicitly prohibited for Version 1.¹

When an implementation prompt is executed, the AI is instructed to create a dedicated feature branch using a strict naming convention that references the module identifier (e.g., `feature/module-1a-authentication`). The AI is expected to commit atomic, logically separated changes, isolating database migrations, backend abstraction logic, API controllers, and frontend components into distinct commits within the feature branch. The repository workflow strictly prohibits the AI from modifying the finalized code of previously approved modules unless the current module explicitly requires an authorized cross-cutting interface update, which must be clearly documented and isolated in the pull request.¹

Automated CI/CD pipelines are triggered immediately upon pull request creation. These pipelines execute static analysis, linting, unit tests, and cross-module integration tests before human review begins. Database migrations involving PostgreSQL Row-Level Security require specialized parsing in the CI pipeline; the workflow dictates that migration scripts must be scanned to ensure no table is created without an associated isolation policy mapping to the active session context.¹ Furthermore, distinct and fully isolated development, staging, and production environments must be maintained, with the workflow enforcing that provider credentials (such as Razorpay or Resend) are injected securely via environment variables and never committed to source control.¹

5. Phase Workflow

The execution of the Crewly project is divided into distinct operational phases, requiring formal transition rituals between each to ensure structural and architectural integrity. The transition from the Platform Foundation phase to the Business Module phase represents the most critical juncture in the development lifecycle. During the Platform Foundation phase, the AI constructs the internal capabilities—Authentication, Organization Management, RBAC, Billing,

Notifications, Audit Logging, and File Storage—that are never customer-subscribable on their own, but which every subsequent module relies upon.¹

Before advancing to the MVP Business Modules (starting with Employee Management), a phase-gate review is conducted. This review ensures that the unified REST Application Programming Interface (API) surface is fully stable, properly versioned under `/api/v1/`, and that the OpenAPI specifications are accurately generated.¹ The phase workflow dictates that this OpenAPI documentation serves as the authoritative machine-readable contract for all future integrations, including the future Marketplace extension surface.

When advancing from the MVP Cutoff at Module 9 (Payroll) into the Post-MVP Roadmap modules, cross-module data dependencies become significantly more complex. Consequently, the phase workflow requires continuous validation against the locked Module Dependency Matrix.¹ Any attempt by the AI to violate this matrix during a phase execution—for instance, by allowing the CRM module to duplicate the Core Directory's Employee entity rather than referencing it—will result in an immediate rollback of the phase branch. Phase workflows also include mandatory architectural consistency checks to ensure that cross-cutting concerns, such as localization readiness and standard error envelopes carrying correlation IDs, have not degraded over time.¹

6. Module Workflow

The lifecycle of a single module's implementation follows a highly controlled, iterative cycle designed to enforce the "RLS first, RBAC second" mandate.¹ The module workflow begins with context assembly. The supervising engineer compiles the approved eighteen-point design alongside the Prompt Structure Standard, injecting this comprehensive context into the Antigravity agent.

The AI proceeds with code generation, adhering to a strict internal sequence. It must initially focus on the data layer, generating the Supabase PostgreSQL database migrations, defining the schemas, and applying the non-negotiable Row-Level Security policies to establish the tenant data boundaries before any application logic is written.¹ Following the secure data layer, the AI constructs the backend architecture: the provider abstractions (if external services are involved), the business logic controllers, and the versioned REST API endpoints. The frontend components, utilizing Vite, React, and Tailwind CSS, are generated last, explicitly designed to consume the newly created API endpoints while adhering to WCAG 2.1 AA accessibility standards.¹

Once code generation is complete, the AI must synthesize unit and integration tests. Unit testing is heavily mandated for business-rule-dense logic, such as RBAC's deny-wins resolution, Leave Management's balance-transaction ledger, and Payroll's calculation engine.¹ The automated build and test cycle then runs. If failures occur, the AI is permitted to execute self-correction loops within the branch, provided it strictly adheres to the frozen architecture and does not attempt to bypass failing tests by altering foundational constraints. Once the pipeline succeeds, the module formally enters the human review queue.

7. Quality Gates

No module is permitted to merge into the main repository without passing an exhaustive series of automated and manual quality gates. These gates heavily enforce the nineteen Cross-Cutting Platform Architecture standards locked in Phase 6 of the Blueprint.¹ Automated static analysis tools enforce coding standards, specifically verifying the proper sanitization of inputs to prevent injection attacks and confirming the correct implementation of the platform-wide standard error envelope, which must always carry a correlation identifier for end-to-end trace observability.¹

The most critical automated quality gate is the Row-Level Security validation script. This script scans every newly generated database table to ensure the presence of the `organization_id` column and the active enforcement of tenant isolation.¹ Furthermore, a dependency matrix verification tool analyzes the generated codebase to ensure that the AI has not introduced unauthorized external software development kits or established direct couplings. For example, the tool verifies that the Notification Framework communicates exclusively with the internal Notification Provider abstraction rather than directly importing Resend libraries into business logic controllers.¹

Additional quality gates evaluate API standards, ensuring that additive changes are backward-compatible and that rate limiting, utilizing 429 status codes with Retry-After headers, is appropriately configured to protect shared infrastructure.¹ The gates also enforce the caching strategy, ensuring no RBAC allow/deny decision is cached beyond the strict 60-second maximum propagation window, and verify that all user-facing strings are externalized to translation resource files to satisfy the internationalization (i18n) readiness requirement.¹ Any module failing a quality gate is immediately rejected and sent back to the AI, with the automated error logs serving as the sole feedback mechanism for correction.

8. Human Review Guide

The human review process acts as the final and most vital defense against architectural drift, AI hallucination, and the degradation of global principles. The supervising senior engineer must evaluate the pull request primarily for strict adherence to the frozen Blueprint, treating the AI's output with high skepticism. The reviewer must manually trace the data flow to check for Row-Level Security leaks, ensuring that no database query bypasses the active session context or attempts to fetch data across tenant boundaries.¹

The reviewer is required to execute a strict checklist verifying provider abstraction compliance. They must confirm that business logic controllers never instantiate external clients directly. Hard-delete violations must be scrutinized meticulously; the reviewer must inspect every endpoint that handles data removal to ensure that it exclusively implements soft-delete flags, archiving state transitions, or status lifecycle updates.¹ The UI/UX consistency is evaluated manually to guarantee that the React frontend meets the accessibility targets, remains mobile-responsive, and appropriately utilizes standard components.¹

A critical aspect of the human review is detecting unauthorized parallel logic. If the AI attempts

to write an independent notification dispatcher rather than routing messages through Module 3, or if it bypasses Module 4 by writing security-critical logs to a standard operational logger instead of the immutable Audit Logging interface, the reviewer must reject the pull request entirely.¹ The reviewer will then issue a corrective prompt, reminding the AI of the centralized interface rules and forcing a rewrite of the offending logic.

9. Antigravity Interaction Rules

Interacting with the Google Antigravity agent requires rigid linguistic and operational discipline. The supervising engineer must communicate in a purely directive, non-collaborative tone. The rules explicitly forbid the engineer from asking the AI for design opinions, architectural improvements, or technology substitutions.¹ If the AI generates an output containing inline comments or suggestions that advocate for a deviation from the Blueprint due to a perceived inefficiency or a modern framework trend, the engineer must command the AI to revert to the explicitly defined architecture immediately.

If the AI experiences a context window failure and forgets a global architectural principle—such as attempting to hardcode a timezone rather than storing it as UTC-internal¹—the engineer must halt generation. The interaction rules require the engineer to refresh the context by re-injecting Prompt 00 alongside the current task definition. Course corrections must be definitive. The rules absolutely prohibit the engineer from authorizing temporary workarounds, technical debt, or "hacky" solutions to bypass a complex implementation challenge. The AI must be forced to solve the engineering problem strictly within the immovable constraints of the Enterprise SaaS Blueprint v3.¹

10. Risk Management

The primary risks inherent in utilizing an autonomous coding agent for complex enterprise software development involve architectural drift, unauthorized data duplication, and the subtle introduction of insecure defaults. Risk management protocols dictate that every generated file must be aggressively audited against the "Never a Second Source of Truth" mandate.¹

To prevent AI drift, the supervising engineer must verify that the AI is not creating redundant database columns to store data already owned by another module. For instance, when constructing the Attendance module, the AI might attempt to duplicate the employee's name and email; the risk management protocol requires this to be caught and corrected so that only the immutable Employee ID is referenced and joined at the presentation layer.¹ Token limit management is also a recognized risk; extremely complex modules like Finance or Payroll must be carefully prompted in sub-sections to ensure the AI's context window does not truncate critical business rules.¹

Furthermore, the "fail closed" rule must be explicitly tested during the review phase.¹ Reviewers must submit malformed requests, unauthorized access attempts (such as a Manager attempting to access Super Admin routes), and out-of-bounds context payloads to the generated application. This ensures the system gracefully defaults to a secure denial rather than failing open, bypassing RBAC, or crashing the shared Supabase infrastructure.

11. Final Prompt Roadmap

The following subsections define the exhaustive, sequential roadmap for every prompt required to construct the Crewly platform. Each step outlines the exact specifications the AI must achieve, acting as the operational manual for generating the actual prompts.

11.1 Prompt 00: Master Project Prompt

The Master Project Prompt establishes the perpetual context for the entire engagement. It is not an implementation prompt but the architectural constitution. It dictates the pricing schema extensibility, the multi-tenancy model, the technology stack, and the non-negotiable golden rules of development.¹

Attribute	Specification
Purpose	Establish permanent context, global architectural principles, and technology stack.
Dependencies	None.
Inputs	Blueprint v3 Product Vision, BRD, System Architecture, Global Principles.
Outputs	AI acknowledgment of constraints and operational readiness.
Expected Deliverables	No code. Verbal confirmation of rules.
Review Criteria	AI correctly parses the prohibition on redesigns, microservices, and hard deletes.
Completion Requirements	AI state initialized and ready to receive Prompt 00A.

11.2 Prompt 00A: Repository Initialization

This prompt is explicitly restricted to establishing the development foundation. It must not implement any business logic, database tables, or application interfaces. Its sole purpose is to build the environment in which all subsequent modules will operate.¹

Attribute	Specification
Purpose	Initialize repository, CI/CD, Docker, shared packages, and testing framework.
Dependencies	Prompt 00.
Inputs	Tooling requirements (Node.js, React, Vite, Supabase local dev setup, GitHub Actions).
Outputs	Base repository structure, environment configuration files, linting rules.
Expected Deliverables	package.json, Docker Compose file, .github/workflows, ESLint/Prettier configs.
Review Criteria	Strict monolithic structure; distinct staging/prod configurations; zero business logic.
Completion Requirements	Clean CI pipeline run indicating a healthy, empty, linted repository.

11.3 Prompt 01a: Module 1a Authentication

This prompt initiates the Platform Foundation by establishing secure, platform-wide user identity. The AI must construct the authentication layer independent of any organization, handling multi-tenant session switching without requiring re-authentication, and integrating Supabase Auth securely.¹

Attribute	Specification
Purpose	Build the secure front door and identity management system.
Dependencies	Prompt 00A.
Inputs	Blueprint Module 1a specifications (self-service signup, MFA, lockout rules).

Outputs	Supabase Auth integration, user tables, session management logic.
Expected Deliverables	Auth API endpoints, login/signup frontend components, email verification hooks.
Review Criteria	Passwords never stored in plain text; anti-enumeration enforced; Super Admin MFA mandatory and non-configurable.
Completion Requirements	User can sign up, verify email, and log in successfully across multiple sessions.

11.4 Prompt 01b: Module 1b Organization Management

The AI must build the tenant container, ensuring every signed-up user lands in a real tenant. This module establishes the structural lifecycle of an organization and uniquely owns the Module Registry and Activation Model, tracking which downstream modules are active for the tenant.¹

Attribute	Specification
Purpose	Establish the organization as the tenant container and manage internal structures.
Dependencies	Module 1a.
Inputs	Blueprint Module 1b specifications (Structure Nodes, Memberships, Module Registry ownership).
Outputs	Organization tables, RLS policies, ownership transfer workflows.
Expected Deliverables	Org switcher UI, Structure Node builder API, membership lifecycle controllers.
Review Criteria	Module registry must not leak into Billing logic; ownership transfer must be strictly

	atomic.
Completion Requirements	A user can create an organization, invite members, and build a hierarchical structure node.

11.5 Prompt 01c: Module 1c RBAC

The AI is tasked with layering fine-grained access control over the coarse platform roles defined in Module 1b. The AI must create a centralized permission-check interface that all future modules will blindly trust and consume, enforcing fail-closed logic.¹

Attribute	Specification
Purpose	Provide highly configurable access control on top of coarse roles.
Dependencies	Module 1a, Module 1b.
Inputs	Blueprint Module 1c specifications (Custom roles, permission groups, deny-wins logic).
Outputs	RBAC engine, central can(user, action, resource) interface.
Expected Deliverables	Role management UI, default seeded permissions, propagation logic.
Review Criteria	Conflicting grants must strictly resolve to deny; check interface must fail closed on unknown combinations; 60-second max propagation.
Completion Requirements	Custom roles can be created and accurately restrict API endpoint access.

11.6 Prompt 02: Module 2 Subscription & Billing Engine

This prompt requires the AI to construct the centralized enforcement gate for all subscribable

modules. The AI must integrate Razorpay behind a strict Payment Provider abstraction, ensuring zero direct coupling to the gateway, while managing the Trial, Grace, Locked, and Reactivated states.¹

Attribute	Specification
Purpose	Enforce subscription lifecycles and per-module billing.
Dependencies	Module 1b, Module 1c.
Inputs	Blueprint Module 2 specifications (Razorpay v1, extensible pricing schema, read-only-during-Locked).
Outputs	Billing APIs, state machine logic, webhook verification.
Expected Deliverables	Payment abstraction layer, invoice history UI, Locked-state enforcement middleware.
Review Criteria	No payment credentials stored; webhooks cryptographically verified; no business module implements its own subscription check.
Completion Requirements	Organizations gracefully transition to read-only-during-Locked upon trial expiration.

11.7 Prompt 03: Module 3 Notification Framework

The AI must realize the stubbed notification interfaces from prior modules. The instruction mandates a channel-agnostic dispatch service, integrating Resend for email delivery behind a Notification Provider abstraction, while creating an in-app inbox as the durable source of truth.¹

Attribute	Specification
Purpose	Centralize channel-agnostic dispatch services platform-wide.

Dependencies	Module 1a, Module 1c.
Inputs	Blueprint Module 3 specifications (Resend v1, versionable templates, idempotency).
Outputs	Notification Provider interface, template engine, notify() dispatcher.
Expected Deliverables	In-app inbox UI, delivery retry logic, user preference configurations.
Review Criteria	All dispatch calls must be idempotent; security notifications bypass opt-outs; provider abstraction enforced.
Completion Requirements	Prior modules successfully route emails through the new centralized interface.

11.8 Prompt 04: Module 4 Audit Logging

The AI must build the immutable, append-only record system for every sensitive action platform-wide. The prompt explicitly prohibits the AI from exposing any update or delete API methods for audit records, ensuring strict compliance readiness.¹

Attribute	Specification
Purpose	Provide a single, centralized, append-only record of state-changing actions.
Dependencies	Module 1c.
Inputs	Blueprint Module 4 specifications (Structured event ingestion, Correlation IDs).
Outputs	Append-only storage tables, event ingestion interface, CSV export.
Expected Deliverables	Org-scoped trail views, Super Admin

	cross-org viewer, ingestion API.
Review Criteria	Absolutely no write access beyond programmatic append; ingestion cannot block main business threads; Super Admin access logged.
Completion Requirements	All previously built modules emit structured events into the logger successfully via the unified interface.

11.9 Prompt 05: Module 5 File Storage

The final Platform Foundation prompt tasks the AI with centralizing file handling. The AI must integrate Supabase Storage behind a Storage Provider abstraction, ensuring that every uploaded file is strictly isolated by tenant namespaces and accessed only via short-lived signed URLs.¹

Attribute	Specification
Purpose	Centralize tenant-isolated, provider-abstracted file storage.
Dependencies	Module 1c, Module 2, Module 4.
Inputs	Blueprint Module 5 specifications (Supabase Storage v1, signed URLs, soft-delete).
Outputs	Storage Provider interface, metadata tables in Postgres, upload/download handlers.
Expected Deliverables	Immutable File ID referencing system, MIME/size validation logic.
Review Criteria	No raw storage SDK calls permitted in business logic; files never public; metadata cleanly separated from binary objects.

Completion Requirements	Secure upload and signed URL retrieval verified across tenant boundaries.
-------------------------	---

11.10 Prompt 06: Module 6 Employee Management (MVP)

Initiating the MVP business modules, the AI must establish the authoritative Employee entity in the Core Directory. This prompt commands the AI to bridge the system access Membership concept with the human resources Employee concept without duplicating data, utilizing immutable internal IDs.¹

Attribute	Specification
Purpose	Own the authoritative Employee record consumed by downstream modules.
Dependencies	Module 1b, Module 1c, Module 5, Module 2.
Inputs	Blueprint Module 6 specifications (Employment lifecycle, custom fields, structure node assignment).
Outputs	Employee schema, field-level permission structures, directory API.
Expected Deliverables	Employee profile UI, self-service view, offboarding workflow.
Review Criteria	Employee ID must be immutable; no hard deletions upon termination; records never duplicated.
Completion Requirements	Employees can be created, linked to Memberships, and transitioned to Terminated state while retaining history.

11.11 Prompt 07: Module 7 Attendance (MVP)

The AI is instructed to build the time-and-presence data foundation. The prompt emphasizes that the attendance ledger must handle corrections immutably, preserving the original entry

alongside the corrected value, and mandates UTC-internal storage for all timestamps.¹

Attribute	Specification
Purpose	Track employee presence via a structured, auditable record.
Dependencies	Module 6, Module 1c, Module 2.
Inputs	Blueprint Module 7 specifications (Configurable entry methods, regularization windows, Attendance status).
Outputs	Attendance ledger tables, clock-in/out APIs, policy configuration engine.
Expected Deliverables	Manager approval UI, summary views, missing-entry flagging logic.
Review Criteria	UTC-internal calculations strictly enforced; policies apply prospectively only; corrections preserve original values.
Completion Requirements	Accurate recording of time inputs and manager-approved corrections reflecting properly in the ledger.

11.12 Prompt 08: Module 8 Leave Management (MVP)

The AI must construct the authoritative source of leave decisions. The prompt requires the AI to implement an integration interface that writes "On Leave" statuses directly into the Attendance module, ensuring approved leaves override raw attendance data automatically, while balances remain derived from transactions.¹

Attribute	Specification
Purpose	Manage the full leave lifecycle and interface directly with Attendance.
Dependencies	Module 6, Module 7, Module 1c, Module 2.

Inputs	Blueprint Module 8 specifications (Leave policies, accrual math, balance tracking).
Outputs	Leave request workflows, balance transaction tables, Attendance status webhook.
Expected Deliverables	Leave calendar UI, multi-step approval chains, automated balance deduction.
Review Criteria	Balances must derive from immutable transactions, not direct edits; no negative balances by default; policy changes prospective.
Completion Requirements	Approved leave correctly propagates to Attendance and immutably deducts from derived balances.

11.13 Prompt 09: Module 9 Payroll (MVP Cutoff)

Reaching the MVP Cutoff, the AI must calculate employee compensation by exclusively consuming data from the Employee, Attendance, and Leave modules. The prompt strictly prohibits the AI from allowing the Payroll module to override or recalculate the upstream data sources, and mandates strict field-level RLS for compensation data.¹

Attribute	Specification
Purpose	Consolidate HR data into accurate, auditable, immutable payroll runs.
Dependencies	Modules 6, 7, 8, 1c, 5, 2.
Inputs	Blueprint Module 9 specifications (Salary structures, immutable finalized runs).
Outputs	Payroll calculation engine, payslip generator, Draft-to-Finalized state machine.

Expected Deliverables	Payslip UI, approval workflows, field-level access control application.
Review Criteria	Finalized runs are absolutely immutable; post-finalization corrections require new entries; financial data highly restricted.
Completion Requirements	Accurate calculation of gross/net pay resulting in an immutable Finalized Run with generated payslips.

11.14 Prompt 10: Module 10 CRM (Roadmap)

The AI must establish the Contact and Account shared entities for the Core Directory. The prompt dictates that these entities must be architected for reuse by future Help Desk, Finance, and Projects modules without data duplication, and that lead conversions maintain an append-only interaction history.¹

Attribute	Specification
Purpose	Manage prospective and existing customer relationships via a structured pipeline.
Dependencies	Module 1c, Module 5, Module 2, Module 6.
Inputs	Blueprint Module 10 specifications (Configurable pipelines, interaction logs, Contact/Account ownership).
Outputs	Lead/Deal workflows, Contact/Account schema, activity append-only logs.
Expected Deliverables	Deal boards, conversion logic, interaction history UI.
Review Criteria	A Converted Lead cannot be re-converted; activity logs must be immutable; Contact/Account established as shared entities.

Completion Requirements	Complete lead-to-close tracking with properly linked Account entities and full interaction history.
-------------------------	---

11.15 Prompt 11: Module 11 Projects (Roadmap)

The AI is tasked with building internal and client-facing project tracking. The prompt must explicitly mandate that Project Tasks remain completely structurally distinct from Help Desk Tickets, preventing the AI from merging the two entities to save development time.¹

Attribute	Specification
Purpose	Manage project hierarchies, milestones, and raw time tracking.
Dependencies	Module 1c, Module 5, Module 2, Module 10, Module 6.
Inputs	Blueprint Module 11 specifications (Task hierarchies, Project Membership scoping, optional Account linkage).
Outputs	Project/Task tables, time-entry APIs, comment feeds.
Expected Deliverables	Kanban/List views, client linkage to CRM Accounts, status configurations.
Review Criteria	Project Membership grants scoped visibility distinct from global RBAC; Project Tasks never merge with Help Desk Tickets.
Completion Requirements	Workable task tracking linked to specific Employee assignments and CRM Accounts with full time-logging capability.

11.16 Prompt 12: Module 12 Help Desk (Roadmap)

The AI must build a customer support request workflow. The critical directive for this prompt is

the structural separation of internal notes from external-facing responses. The AI must enforce this at the database entity level, ensuring it is mathematically impossible to expose an internal note externally.¹

Attribute	Specification
Purpose	Manage internal and external support requests using CRM shared entities.
Dependencies	Modules 10, 6, 1c, 5, 3, 4, 2.
Inputs	Blueprint Module 12 specifications (Ticket lifecycle, basic SLA tracking, internal vs. external structural boundary).
Outputs	Ticket tables, distinct Internal Note and External Response tables, assignment APIs.
Expected Deliverables	Agent queue UI, response editor, SLA breach visual indicators.
Review Criteria	Internal notes mathematically impossible to expose to external requesters; tickets never merge with Project Tasks.
Completion Requirements	Tickets accurately routed, worked, and resolved with pristine internal note isolation and accurate SLA flagging.

11.17 Prompt 13: Module 13 Inventory (Roadmap)

The AI must construct the platform's authoritative stock ledger. The instruction strictly forbids the AI from making stock levels directly editable; quantities must always be mathematically derived from the immutable movement ledger (Receipt, Issue, Transfer, Adjustment).¹

Attribute	Specification
Purpose	Track stock items and physical locations via an auditable movement ledger.

Dependencies	Modules 6, 5, 3, 4, 1c, 2.
Inputs	Blueprint Module 13 specifications (SKU management, Movement events, Reorder logic).
Outputs	Item/Location schemas, Movement transaction engine, Reorder point logic.
Expected Deliverables	Movement ledger UI, real-time stock calculation APIs, low-stock notifications.
Review Criteria	Stock quantities derived exclusively from movements; corrections require offsetting entries; negative stock blocked by default.
Completion Requirements	Stock moves between locations immutably, accurately reflecting current on-hand quantities without direct edits.

11.18 Prompt 14: Module 14 Finance (Roadmap)

The AI must consolidate financial record-keeping without embedding accounting logic into upstream modules. The prompt instructs the AI to pull finalized snapshots from Payroll, CRM, Projects, and Inventory, applying a Weighted Average valuation for inventory cost without recalculating original data.¹

Attribute	Specification
Purpose	Act as the single source of financial truth consuming upstream module data.
Dependencies	Modules 10, 11, 9, 13, 6, 1c, 5, 3, 4, 2.
Inputs	Blueprint Module 14 specifications (Invoicing, Expenses, Weighted Average COGS, read-only consumer rules).
Outputs	Financial ledger tables, expense approval

	workflows, period summary logic.
Expected Deliverables	Invoice generation UI, expense upload via File Storage, Chart of Accounts config.
Review Criteria	Finance strictly consumes read-only data; it never mutates upstream module state; financial records follow immutable transaction models.
Completion Requirements	Invoice lifecycle managed and inventory valuation consistently matches movement ledgers using weighted average calculations.

11.19 Prompt 15: Module 15 Analytics & Reports (Roadmap)

The AI must build a cross-module reporting engine. The prompt explicitly prohibits the AI from creating a secondary data store; Analytics must read exclusively through each owning module's stable reporting interface, evaluating RBAC rules dynamically at run-time.¹

Attribute	Specification
Purpose	Provide read-only aggregation and visualization across all business modules.
Dependencies	Every preceding business module, Modules 1c, 3, 4, 5, 2.
Inputs	Blueprint Module 15 specifications (Ad-hoc builder, dashboard widgets, no duplicate storage).
Outputs	Report definition schema, cross-module querying APIs (no raw SQL), CSV exporter.
Expected Deliverables	Dashboard UI, scheduled report dispatcher, RBAC-scoped data filtering.

Review Criteria	Viewers cannot bypass module-level RBAC through analytics dashboards; no raw database access permitted.
Completion Requirements	User can generate a custom cross-module report reflecting real-time, RBAC-filtered data without duplicate storage.

11.20 Prompt 16: Module 16 Platform AI Layer (Shared Capability)

The AI is tasked with creating a shared, provider-agnostic AI capability layer. The AI must build an abstraction interface (defaulting to a self-hosted Ollama philosophy) that never persists calling-module content beyond operational metering metadata, ensuring the AI Layer never becomes a secondary source of truth.¹

Attribute	Specification
Purpose	Provide single provider-agnostic AI capability layer for downstream consumption.
Dependencies	Modules 1c, 2, 4, 3.
Inputs	Blueprint Module 16 specifications (Capability checks, token metering, stateless processing).
Outputs	AI Provider abstraction, generate() interface, usage tracking tables.
Expected Deliverables	Super admin configuration UI, context assembly APIs, content safety guardrails.
Review Criteria	The AI layer never directly queries the database; it processes explicitly provided context only; it never persists business content.
Completion Requirements	Business modules can trigger text generation/summarization via the stable

	interface while maintaining data privacy.
--	---

11.21 Prompt 17: Module 17 Marketplace (Shared Capability)

The final module prompt requires the AI to surface the internal Module Registry into a browsable storefront. The prompt explicitly states that Marketplace never owns activation logic or pricing; it strictly invokes Organization Management and Billing, acting solely as a discovery layer.¹

Attribute	Specification
Purpose	Provide a discovery and activation storefront for first-party modules and future extensions.
Dependencies	Module 1b, Module 2, Module 1c, Module 4, All Business Modules.
Inputs	Blueprint Module 17 specifications (Dependency Matrix surfacing, extension architecture placeholder).
Outputs	Catalog listing schemas, discovery APIs, activation hand-off controllers.
Expected Deliverables	Marketplace browsing UI, dependency validation displays.
Review Criteria	No third-party execution implemented in v1; activation logic remains centralized in Billing; dependency matrix strictly enforced.
Completion Requirements	Owner can discover, validate dependencies, and activate a new module via the unified storefront interface.

12. Project Completion Workflow

The final stage of the Crewly AI implementation involves executing the platform finalization

prompts. The Project Completion Workflow shifts the Antigravity agent's focus entirely from feature generation to holistic system validation and deployment staging.

Prompt 18 commands the AI to execute sweeping cross-module integration tests. The AI must verify that the intricate, cross-module contracts—such as Leave overriding Attendance, Inventory valuation accurately feeding Finance, and CRM Accounts seamlessly linking to Project Tasks—operate without failure under stress.¹ Furthermore, the AI must script end-to-end user journeys that test the core commercial flow: signup, trial activation, organization creation, module enablement via Marketplace, and the first business action within the MVP modules.¹

Prompt 19 strictly tasks the AI with automated security penetration testing and an exhaustive Row-Level Security validation audit across the complete, 17-module schema. The AI must simulate hostile, multi-tenant cross-talk to guarantee that the primary database isolation boundaries are impenetrable and that the central RBAC interface fails closed when presented with unknown permutations.¹

Finally, Prompt 20 drives deployment staging. It demands the AI generate the final OpenAPI specifications representing the unified API surface, finalize all developer-facing README documentation, and assemble the production environment variable configurations.¹ The supervising engineer validates the complete audit trail, ensuring that from Prompt 00 to Prompt 20, the frozen architecture remained entirely inviolate, resulting in a production-ready, globally scalable Enterprise SaaS platform.

Works cited

1. Prompt_00_Master_Project_Prompt.md